

# Handwriting-Based Person Identification via Combined Texture and Graphological Features

Melih Coşgun, Orkhan Karimli, Omar Hamdache and Nurşize Turan  
*Department of Computer Science, Bilkent University*

**Abstract**—This project proposes a computer vision pipeline for identifying individuals based on offline handwriting samples. The system integrates texture-based features, specifically Histogram of Oriented Gradients (HOG) and Bag of Visual Words (BoVW), with handcrafted graphological features such as baseline angle, slant angle, curvature, and word spacing. Dimensionality reduction was performed using Principal Component Analysis (PCA) before classification via a Nearest Neighbor search. Experimental results demonstrate that the combination of texture and graphological features yields a Top-1 accuracy of 70% and a Top-10 accuracy of 90% on a subset of 100 users. While accuracy declines as the user pool expands to 352, the high retrieval rates suggest the system is effective for ranking-based identification tasks.

## I. INTRODUCTION

Handwriting identification remains a significant challenge in computer vision due to the high variability of intraclass writing styles and the subtlety of interclass differences. Traditional approaches often rely solely on global texture analysis or local structural features. In this work, we aim to enhance identification robustness by combining these two paradigms.

Our approach utilizes a pipeline given in Figure 1 below, that extracts multiple feature sets from input images. We employ HOG [1] and BoVW[2] to capture the general texture and stroke distribution of the handwriting. These are augmented by specific graphological metrics: baseline angle (writing direction), slant angle (letter tilt), curvature (sharpness), and word spacing. By concatenating these feature vectors and applying PCA for dimensionality reduction, we create a compact representation of a writer's style. The system is evaluated using the IAM dataset [3], testing its scalability and accuracy against varying numbers of users.

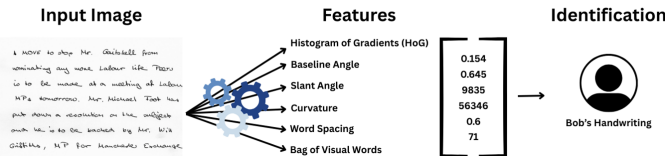


Fig. 1. The basic pipeline of our writer identification approach

## II. RELATED WORK

Many recent systems combine structural (graphological) approaches and texture-based approaches to improve robustness for writer identification tasks.

### A. Structural-Based and Texture-Based Approaches

Structural methods aim to capture writer-specific motor characteristics through explicit geometric features such as baseline stability and slant. Robust extraction often requires accurate segmentation.

Texture-based methods avoid explicit segmentation and are largely language-independent. The Bag of Visual Words (BoVW) model [2] represents handwriting by clustering local descriptors into visual vocabularies, while Histogram of Oriented Gradients (HOG) [1] captures stroke orientation distributions. These descriptors effectively encode stylistic characteristics such as curvature and angularity without precise stroke extraction.

### B. Comparative Baselines

We compare our hybrid graphology-texture approach with two representative methods from the literature:

- **Chahi et al. [4]:** A texture-based framework that treats handwritten connected components as texture images. It extracts features using Local Binary Patterns (LBP), Local Ternary Patterns (LTP), and Local Phase Quantization (LPQ), followed by histogram sequence concatenation and classification using a Nearest Neighbor (1-NN) classifier with a dissimilarity measure.
- **Durou et al. [5]:** A hybrid approach that combines statistical Oriented Basic Image (OBI) features with a model-based grapheme codebook. It employs Kernel Principal Component Analysis (KPCA) for dimensionality reduction and uses a k-Nearest Neighbor (k-NN) classifier for identification.

## III. METHODOLOGY

*a) Graphology Based Features:* We extract a set of structural features inspired by graphology to capture the writer's motor patterns. These features include baseline orientation, handwriting slant, stroke sharpness, and word spacing.

**Baseline and Slant:** To estimate the baseline angle, we first identify the bottom-most ink pixels in each column of the binary image. To ensure robustness against outliers like descenders, we fit a linear model to these points using the RANSAC (Random Sample Consensus) algorithm. The baseline feature  $f_{base}$  is defined as the slope angle of this robustly fitted line. Figure 2 illustrates the detected baseline and the computed angle from the fitted line for a sample handwriting image.



Fig. 2. Illustration of baseline detection. The solid line indicates the baseline estimated using the RANSAC algorithm, which robustly fits the dominant orientation while ignoring outliers caused by descenders (e.g., ‘g’, ‘y’, ‘j’).

For the slant angle, we apply Canny edge detection followed by the Probabilistic Hough Transform to detect linear strokes. We filter for lines with orientations between  $45^\circ$  and  $135^\circ$  to isolate vertical-like strokes. The slant feature  $f_{slant}$  is computed as the median deviation of these stroke angles from the vertical axis ( $90^\circ$ ). Figure 3 illustrates the detected slant lines and the computed slant offset for a sample handwriting image.

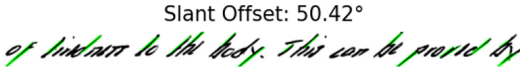


Fig. 3. Example of slant detection. Green lines indicate the detected strokes used to compute the median slant offset of  $50.42^\circ$ .

**Stroke Sharpness:** We quantify the “pointiness” of the handwriting by analyzing the ratio of high-frequency turning points to the total stroke length. We first skeletonize the text using iterative morphological thinning to obtain a thickness-invariant representation. We then detect corner keypoints using the Harris-Laplace detector, which provides scale invariance. The sharpness score  $S$  is calculated as:

$$S = \alpha \cdot \frac{N_{corners}}{N_{skeleton}} \quad (1)$$

where  $N_{corners}$  is the count of valid keypoints,  $N_{skeleton}$  is the total number of pixels in the text skeleton, and  $\alpha$  is a scaling factor.

**Word Spacing:** To measure the spatial arrangement of words, we employ the scale space technique proposed by Manmatha et al [6]. An anisotropic Gaussian filter kernel is applied to the image to merge intra-word characters while preserving inter-word gaps:

$$K(x, y) = \frac{x^2 - \sigma_x^2}{2\pi\sigma_x^5\sigma_y} \exp\left(-\frac{x^2}{2\sigma_x^2} - \frac{y^2}{2\sigma_y^2}\right) + \dots \quad (2)$$

After thresholding the filtered image, word candidates are segmented. We cluster these candidates into lines using the DBSCAN algorithm based on vertical overlap. Finally, we compute the horizontal distances between adjacent word bounding boxes. The feature vector includes the mean ( $\mu_{gap}$ ), median ( $m_{gap}$ ), and standard deviation ( $\sigma_{gap}$ ) of these inter-word spacings.

**b) Texture-Based Features:** To capture the textural properties of the handwriting, we employ Histogram of Oriented Gradients (HOG) [1] and Bag of Visual Words (BoVW) [2].

For HOG, we follow the configuration proposed in [1]. As the input images vary in width, the resulting HOG feature maps have variable dimensions, which is incompatible with

standard classifiers. To obtain a fixed-size representation, we compute the mean  $\mu_H$  and standard deviation  $\sigma_H$  of the feature map across the spatial dimensions. Let  $H \in \mathbb{R}^{h \times w \times b}$  be the HOG feature map, where  $h$  is the fixed height,  $w$  is the variable width, and  $b$  is the number of orientation bins. The pooled feature vector  $f_{HOG}$  is given by:

$$f_{HOG} = [\mu_H, \sigma_H] \quad (3)$$

where the statistics are computed over the width dimension, resulting in descriptors of size  $h \times b$ .

For BoVW, we first extract Scale-Invariant Feature Transform (SIFT) descriptors [7] from the training set. These descriptors are clustered using K-Means to form a codebook of visual words  $C = \{c_1, c_2, \dots, c_K\}$ . For any given image, we extract local descriptors  $D = \{d_1, d_2, \dots, d_M\}$  and map each descriptor to the nearest visual word. The image is represented by a histogram  $h_{BoVW} \in \mathbb{R}^K$ , where the  $i$ -th bin counts the occurrences of word  $c_i$ :

$$h_{BoVW}(i) = \sum_{j=1}^M \mathbb{I}(\text{NN}(d_j) = c_i) \quad (4)$$

where  $\mathbb{I}(\cdot)$  is the indicator function and  $\text{NN}(d_j)$  returns the nearest cluster center for descriptor  $d_j$ . The final feature vector is  $L_2$  normalized:  $f_{BoVW} = h_{BoVW} / \|h_{BoVW}\|_2$ .

**c) Combining the Features:** The final feature representation is constructed by concatenating the texture-based features ( $f_{HOG}, f_{BoVW}$ ) and the graphological features ( $f_{graph}$ ):

$$f_{combined} = [f_{HOG}, f_{BoVW}, f_{graph}] \quad (5)$$

To ensure that each feature contributes equally, we perform Z-score normalization. Given a feature vector  $x$ , the normalized vector  $\hat{x}$  is computed as:

$$\hat{x} = \frac{x - \mu_{train}}{\sigma_{train}} \quad (6)$$

where  $\mu_{train}$  and  $\sigma_{train}$  denote the mean and standard deviation vectors computed from the training set, respectively, and the division is performed element-wise. Finally, we apply Principal Component Analysis (PCA) [8] to reduce the dimensionality of the concatenated vector, retaining the components that explain the majority of the variance. During testing, we scale and project the test features using the parameters derived from the training set.

## IV. EXPERIMENTS AND RESULTS

**a) Experimental Setup:** We utilized the IAM Handwriting Database, which comprises 1539 handwritten documents from 657 distinct writers [3]. For our experiments, we selected a subset of 352 users, using 10 text lines per user. For each user, the data was split into 8 sample for training, 1 for validation, and 1 for test sets, where the validation set was used for hyperparameter tuning and the test set for final evaluation. For identification, we employed a K-Nearest Neighbors (KNN) classifier with  $k = 1$  and Euclidean distance as the metric. We evaluated the system’s performance using Top-k accuracy metrics (Top-1, Top-3, and Top-10).

b) *Preprocessing*: To isolate handwriting samples for feature extraction, we developed a preprocessing pipeline that segments full-page scanned forms into individual text lines and normalizes their appearance.

**Line Segmentation**: First, we binarize the grayscale image using Otsu’s method to separate ink from the background. To merge discontinuous character strokes into connected text blobs, we apply a morphological dilation with a wide rectangular kernel ( $100 \times 5$  pixels). We then extract contours from this dilated image. Contours are filtered to remove noise (bounding box width  $< 400$  or height outside the range  $[20, 100]$ ) and sorted vertically to preserve the reading order.

**Background and Ink Normalization**: Once individual lines are cropped, we normalize the illumination to remove uneven shading. We estimate the background by applying a large Gaussian blur ( $51 \times 51$ ) to the crop and divide the original image by this estimate. To repair small discontinuities in the ink caused by scanning artifacts, we perform a morphological opening operation. Finally, we apply a “soft thresholding” contrast stretch. We compute an optimal threshold  $T$  via Otsu’s method and linearly map the pixel intensities such that values below  $0.6 \cdot T$  become pure black (solid ink) and values above a dynamic upper bound become pure white (clean background), enhancing the stroke edges without creating jagged artifacts.

**Standardization**: To ensure consistency across samples, all segmented lines are resized to a fixed height of 64 pixels. We use bicubic interpolation for upscaling to maintain edge smoothness and area-based resampling for downscaling to prevent aliasing.

c) *Hyperparameter Tuning*: We performed tuning on a subset of 100 users to optimize the Bag of Visual Words (BoVW) cluster size and Principal Component Analysis (PCA) variance. Results are obtained on the validation set.

- **BoVW Clusters**: We tested cluster sizes ranging from 50 to 3000, by using only BoVW features. The accuracy peaked at 500 clusters (0.45 Top-1 accuracy) and decreased as the cluster count increased (see Figure 4). Therefore, we continued with 500 clusters.
- **PCA Variance**: We evaluated preserved variance ratios from 0.65 to 0.95. The model achieved optimal performance (0.70 Top-1 accuracy) when retaining 90% of the variance.

d) *Feature Importance*: Before testing on the test dataset, we analyze the importance of each of those features that we previously extracted. The goal is to understand whether some features have a negative impact on the effectiveness of the task or whether some features have negligible effect so that we can ignore them. This test is also done on the validation dataset. Figure 5 shows the importance of each of those features alone then their combination.

We can see that HoG features perform best standalone. Using both HoG and BoVW features shows a significant improvement of 7% top-1 accuracy over HoG alone. This shows that even though BoVW standalone did not perform that good, combining it with HoG improves the overall results. Finally we add the vector of graphology based features (size of 6) and evaluate on the validation set again. This time the accuracy improves by only 2% for top-1. The reason why the

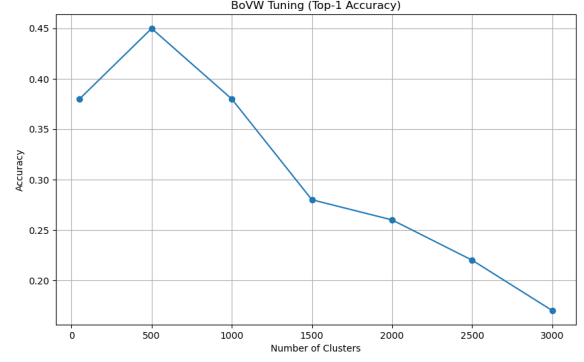


Fig. 4. Hyperparameter tuning for number of clusters in BoVW features. Highest Top-1 accuracy obtained at 500 clusters.

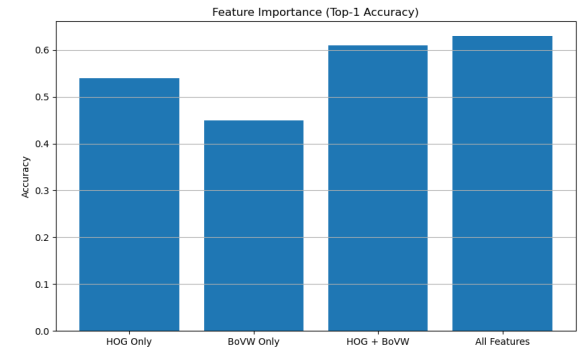


Fig. 5. Feature importance analysis. We try each feature alone, then combine BoVW, HoG, and Graphology based features.

improvement is minimal is that the vector of the graphology based features is of size 6, which is very small compared to the size of 1580 for the other two combined. This makes the effect of those features minimal but noticeable.

e) *Dataset Visualization*: We visualize our dataset on a t-SNE plot to project the high-dimensional feature representations into a two-dimensional space for qualitative analysis. The plot can be seen in Figure 6. We show the samples for writer id 274 as an example in this plot.

Note that due to the high number of features it is a bit difficult to clearly separate the clusters from each other since the used colors are very close to each other due to the limited color range. That’s why we isolate each user individually to better visualize the clusters.

f) *Final Results*: After doing hyperparameter finetuning on the validation dataset and finding the best values for the hyperparameters, we perform a final test of our approach on the test dataset. We also compare our approach with related work. Since we model this as a retrieval task we use the top-1, top-3, and top-10 metrics to evaluate our method. This is also compatible with the literature like Durou et al. [5] and Chahi et al. [4]. The final results are shown in Figure 7.

The first thing that can be noticed in the plot is that the number of users (writers) in the dataset negatively affects the accuracies. For example, top-10 accuracy drops from 0.94 to

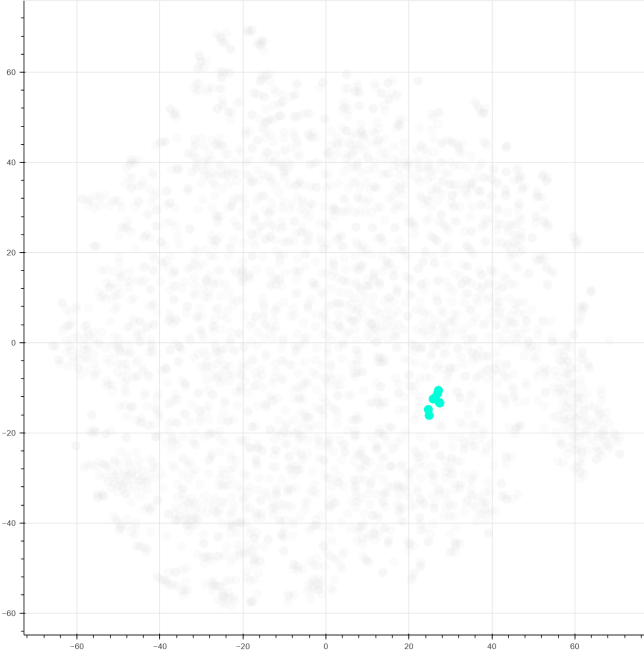


Fig. 6. t-SNE plot the samples corresponding to the writer with id 274. Note that all the other points were grayed out for improved visibility since we have 352 writers.

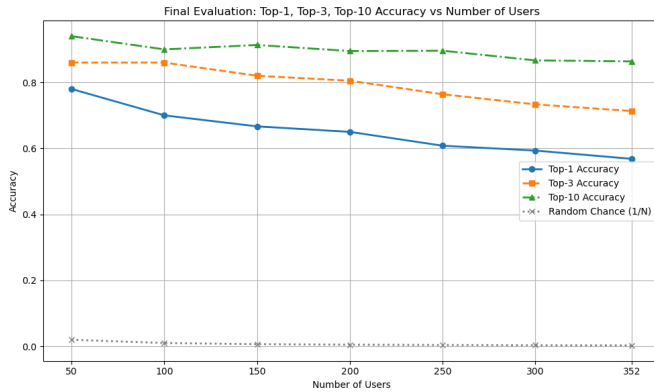


Fig. 7. Final results vs number of users for top-1, top-3, top-7, and top-10 accuracies

0.86 when the users increase from 50 to 352. This is the case since the number of outliers and imperfect clusters in the feature space becomes higher. One way to make the model more robust is to increase the number of samples per class (currently fixed to 8 training). This will allow the clusters to be more dense and therefore create a more defined set of clusters where few outliers won't be that influential.

Finally, we compare with related work to better understand the relevance of our method (see Table I).

We can see that our method performs on a comparable level for the top-10 accuracy metric. While we are noticeably far behind with the top-1 accuracy. One reason might be that the related works use unbalanced datasets, which makes the evaluation more biased to the more common classes, which will also have a higher accuracy due to the nature of the method that utilizes K-NN for classification. We also think that

TABLE I  
COMPARISON OF OUR RESULTS WITH THE LITERATURE.

| Author       | # of Writers | Top-1 | Top-3 | Top-5 | Top-10 |
|--------------|--------------|-------|-------|-------|--------|
| Chahi et al. | 657          | 97.81 | 99.51 | 100   | -      |
| Durou et al. | 650          | 87.96 | -     | 91.34 | 94.28  |
| <b>Ours</b>  | 100          | 70    | 86    | 87    | 90     |
| <b>Ours</b>  | <u>352</u>   | 56.81 | 71.30 | 77.55 | 86.36  |

this big difference between top-1 and top-10 accuracy shows that even though our method succeeds in making the points from the same class land close to each other on the feature space, it still suffers from different classes not being separated enough where some outliers land among the clusters of other classes. This caused the top-1 accuracy to significantly drop as it can be seen in Table I.

## V. IMPLEMENTATION DETAILS

For baseline angle, slant angle, and sharpness features, we implemented the main algorithms, while utilizing key functions from libraries. For example, we used Scikit-Learn for the RANSAC regressor and OpenCV for Thresholding, Morphological primitives, the Hough Transform, Harris-Laplace corner detector, and Canny edge detector.

For word spacing we use the word detection method suggested by Manmatha et al. [6]. We directly use the code implementation for that from github at githubharald/WordDetector. We added our own word spacing calculation logic and also feature vector calculation since the original code only detects words.

When preprocessing the dataset, all image manipulations (e.g., morphological transformations, contour finding, thresholding) used functionality from the OpenCV Python package.

For Z-score normalization, k-means, KNN, t-SNE [9] and PCA operations, we used the scikit-learn Python package. For SIFT (used in the BoVW implementation), we used the OpenCV package.

For visualizations, we used matplotlib for static figures and Bokeh for interactive plots (web apps), which are not provided in this report but were useful for inspecting the t-SNE visualizations during the research process.

In general, we implemented the main algorithms, while using libraries for functionalities like SIFT etc.

## VI. CONCLUSION

We successfully implemented a handwriting identification pipeline that synergizes texture-based descriptors with hand-crafted graphological features. Our experiments indicate that while HOG features provide the strongest standalone performance, the addition of BoVW and specific graphological metrics further improves accuracy. The system achieved a promising 70% Top-1 accuracy for 100 users. Although performance drops to 56.81% as the dataset scales to 352 users, the Top-10 accuracy remains high at over 86%, indicating the model's strength in retrieval and short-listing tasks. Future work could focus on advanced deep learning feature extraction to overcome the limitations observed with larger user sets.

## REFERENCES

- [1] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, vol. 1. Ieee, 2005, pp. 886–893.
- [2] G. Csurka, C. Dance, L. Fan, J. Willamowski, and C. Bray, "Visual categorization with bags of keypoints," in *Workshop on statistical learning in computer vision, ECCV*, vol. 1, no. 1-22. Prague, 2004, pp. 1–2.
- [3] U.-V. Marti and H. Bunke, "The iam-database: an english sentence database for offline handwriting recognition," *International journal on document analysis and recognition*, vol. 5, no. 1, pp. 39–46, 2002.
- [4] A. Chahi, Y. El merabet, Y. Ruichek, and R. Touahni, "An effective and conceptually simple feature representation for off-line text-independent writer identification," *Expert Systems with Applications*, vol. 123, pp. 357–376, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S095741741930051X>
- [5] A. Durou, I. Aref, S. Al-Maadeed, A. Bouridane, and E. Benkhelifa, "Writer identification approach based on bag of words with obi features," *Information Processing & Management*, vol. 56, no. 2, pp. 354–366, 2019.
- [6] R. Manmatha and N. Srimal, "Scale space technique for word segmentation in handwritten documents," in *Scale-Space Theories in Computer Vision*, M. Nielsen, P. Johansen, O. F. Olsen, and J. Weickert, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 22–33.
- [7] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International journal of computer vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [8] K. Pearson, "Liii. on lines and planes of closest fit to systems of points in space," *The London, Edinburgh, and Dublin philosophical magazine and journal of science*, vol. 2, no. 11, pp. 559–572, 1901.
- [9] L. v. d. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.